

0.1.4. ALPHABET — les lettres de la police

#include <ALPHABET> : ALPHABET[0] est le A, ALPHABET[25] le Z — le numéro de la lettre, pour poser(), sprite()...

1. Ce qu'il faut comprendre

Ce que c'est : ALPHABET range les lettres de la police dans l'ordre : ALPHABET[0] est le A, ALPHABET[1] le B... ALPHABET[25] le Z. Ce n'est pas un tableau en mémoire : ALPHABET[i] se calcule « 1 + i », et ne coûte rien de plus.

La ligne à écrire : #include <ALPHABET>.

Ce qu'il coûte : une lettre écrite en clair (ALPHABET[7]) grave **16 octets** : son dessin, et rien d'autre. Une lettre calculée (ALPHABET[i]) peut être n'importe laquelle : la police entière part dans la cartouche, **704 octets**.

Essaie : remplace ALPHABET[7] par ALPHABET[i] dans une boucle : regarde la ROM grossir de 700 octets.

On retrouve ALPHABET dans 309 leçons

[0.2](#), [0.2.1](#), [0.2.2](#), [0.5](#), [0.5.1](#), [0.5.2](#), [0.6](#), [0.6.1](#), [0.6.2](#), [0.7](#), [0.7.1](#), [0.7.2](#), [0.8](#), [0.8.1](#), [0.8.2](#), [0.9](#), [0.9.1](#), [0.9.2](#), [0.10](#), [0.10.1](#), [0.10.2](#), [0.12](#), [0.12.1](#), [0.12.2](#), [0.13](#), [0.13.1](#), [0.13.2](#), [0.14](#), [0.14.1](#), [0.14.2](#), [0.15](#), [0.15.1](#), [0.15.2](#), [0.16](#), [0.16.1](#), [0.16.2](#), [0.17](#), [0.17.1](#), [0.17.2](#), [0.18](#), [0.18.1](#), [0.18.2](#), [0.19](#), [0.19.1](#), [0.19.2](#), [0.20](#), [0.20.1](#), [0.20.2](#), [0.21](#), [0.21.1](#), [0.21.2](#), [0.22](#), [0.22.1](#), [0.22.2](#), [0.23](#), [0.23.1](#), [0.23.2](#), [0.24](#), [0.24.1](#), [0.24.2](#), [0.25](#), [0.25.1](#), [0.25.2](#), [0.26](#), [0.26.1](#), [0.26.2](#), [0.27](#), [0.27.1](#), [0.27.2](#), [0.28](#), [0.28.1](#), [0.28.2](#), [0.29](#), [0.29.1](#), [0.29.2](#), [0.30](#), [0.30.1](#), [0.30.2](#), [0.31](#), [0.31.1](#), [0.31.2](#), [0.32](#), [0.32.1](#), [0.32.2](#), [0.33](#), [0.33.1](#), [0.33.2](#), [0.34](#), [0.34.1](#), [0.34.2](#), [0.35](#), [0.35.1](#), [0.35.2](#), [0.36](#), [0.36.1](#), [0.36.2](#), [0.37](#), [0.37.1](#), [0.37.2](#), [0.38](#), [0.38.1](#), [0.38.2](#), [0.39](#), [0.39.1](#), [0.39.2](#), [0.40](#), [0.40.1](#), [0.40.2](#), [0.41](#), [0.41.1](#), [0.41.2](#), [0.42](#), [0.42.1](#), [0.42.2](#), [0.43](#), [0.43.1](#), [0.43.2](#), [0.44](#), [0.44.1](#), [0.44.2](#), [0.45](#), [0.45.1](#), [0.45.2](#), [0.46](#), [0.46.1](#), [0.46.2](#), [0.47](#), [0.47.1](#), [0.47.2](#), [0.48](#), [0.48.1](#), [0.48.2](#), [0.49](#), [0.49.1](#), [0.49.2](#), [0.50](#), [0.50.1](#), [0.50.2](#), [0.51](#), [0.51.1](#), [0.51.2](#), [0.52](#), [0.52.1](#), [0.52.2](#), [0.53](#), [0.53.1](#), [0.53.2](#), [0.54](#), [0.54.1](#), [0.54.2](#), [0.55](#), [0.55.1](#), [0.55.2](#), [0.56](#), [0.56.1](#), [0.56.2](#), [0.57](#), [0.57.1](#), [0.57.2](#), [0.58](#), [0.58.1](#), [0.58.2](#), [0.59](#), [0.59.1](#), [0.59.2](#), [0.60](#), [0.60.1](#), [0.60.2](#), [0.61](#), [0.61.1](#), [0.61.2](#), [0.62](#), [0.62.1](#), [0.62.2](#), [0.63](#), [0.63.1](#), [0.63.2](#), [0.64](#), [0.64.1](#), [0.64.2](#), [0.65](#), [0.65.1](#), [0.65.2](#), [0.65.4](#), [0.66](#), [0.66.1](#), [0.66.2](#), [0.66.4](#), [0.67](#), [0.67.1](#), [0.67.2](#), [0.67.3](#), [0.68](#), [0.68.1](#), [0.68.2](#), [0.68.3](#), [0.69](#), [0.69.1](#), [0.69.2](#), [0.69.3](#), [0.70](#), [0.70.1](#), [0.70.2](#), [0.70.4](#), [0.71](#), [0.71.1](#), [0.71.2](#), [0.72](#), [0.72.1](#), [0.72.2](#), [0.73](#), [0.73.1](#), [0.73.2](#), [0.74](#), [0.74.1](#), [0.74.2](#), [0.75](#), [0.75.1](#), [0.75.2](#), [0.76](#), [0.76.1](#), [0.76.2](#), [0.76.3](#), [0.76.4](#), [0.76.5](#), [0.76.6](#), [0.76.7](#), [0.76.8](#), [0.76.10](#), [0.76.11](#), [0.76.12](#), [0.76.13](#), [0.76.14](#), [0.77](#), [0.78](#), [0.78.1](#), [0.79](#), [0.79.1](#), [0.80](#), [0.80.1](#), [0.80.2](#), [0.81](#), [0.81.1](#), [0.81.3](#), [0.82](#), [0.82.1](#), [0.83](#), [0.83.2](#), [0.84](#), [0.85](#), [0.86.2](#), [0.90.2](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.93](#), [0.93.1](#), [0.93.2](#), [0.93.3](#), [0.93.4](#), [0.93.5](#), [0.93.6](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.95.1](#), [0.96](#), [0.96.1](#), [0.97](#), [0.97.1](#), [0.98](#), [0.98.1](#), [0.99](#), [0.99.1](#), [0.100](#), [0.100.1](#), [0.101](#), [0.101.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.105](#), [0.105.1](#), [0.106](#), [0.107](#), [0.107.1](#), [0.108](#), [144](#), [145](#), [146](#), [147](#), [148](#), [149](#), [150](#), [151](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **197 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         19 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         32 o ← CopierTuiles – 2 lettres sur 43
```

Lettres gravées : HI

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 2    12 o sur place, dans main()
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <ALPHABET> : les lettres de la police ----
// ALPHABET[0] = A, ALPHABET[1] = B, ... ALPHABET[25] = Z.

#include <ALPHABET> // ALPHABET : le numéro de chaque lettre
#include <poser>
```

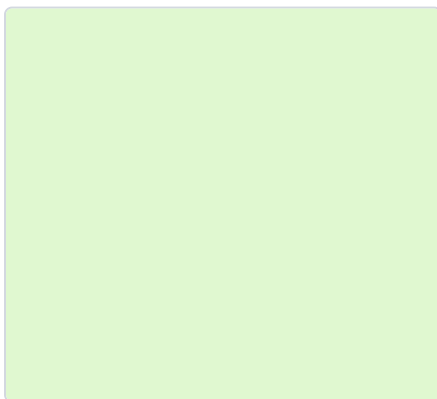
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(8, 8, ALPHABET[7]); // la lettre numéro 7 : H (A=0, B=1, ... H=7)
    poser(9, 8, ALPHABET[8]); // la lettre numéro 8 : I

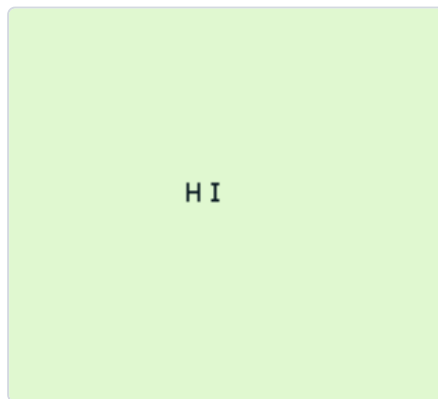
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

HI au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ ALPHABET[7] est un H, ALPHABET[8] un I (« HI »)

Ensuite : **0.2. A, pris dans ALPHABET à l'indice 0** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++